WEEK 8

Math for ML I

Linear Algebra Foundations for Machine Learning

Day 36	**Vectors**	Creation, shapes, addition, scalar multiplication, norms
Day 37	**Dot Product**	Similarity, cosine similarity, linear model scoring
Day 38	**Matrices**	2D arrays, transpose, matrix multiplication rules
Day 39	**Broadcasting**	Implicit expansion, row/column operations, standardization
Day 40	**Matrix Operations Exercise**	Linear transformation pipeline, debugging shapes

LINEAR ALGEBRA IN ML

```
import numpy as np
features = np.array([age, income, clicks])  # Vector: input x
weights = np.array([0.1, 2.0, -0.5])         # Vector: parameters w
score = weights @ features + bias            # Dot product: w·x + b
```

al

DAY 36

Vectors

1D Building Blocks of ML Data and Parameters

OBJECTIVES

- Understand vectors as ordered lists of numbers
- Learn how vectors appear in ML: features, weights, gradients
- Create and inspect vectors in NumPy
- Perform vector addition, scalar multiplication, norms
- Reason about shapes: (n,) vs (n,1) and why it matters

ACTIVITY

Create feature and weight vectors in NumPy.

Perform basic vector operations.

ASSESSMENT

Code submission showing correct vector creation, shapes, and operations.

al

What Vectors Represent in ML

Ordered lists of numbers with ML meaning

Intuitive Meaning

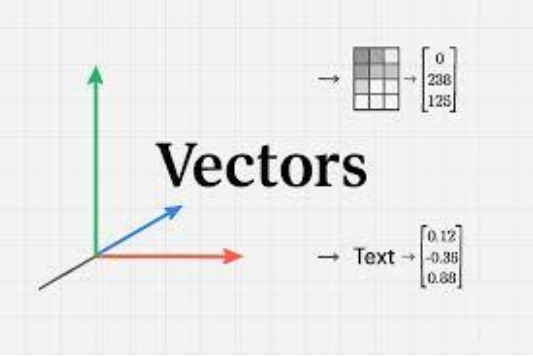
- Ordered list of numbers with multiple components
- Examples: user features [age, income, clicks]
- Model weights for a neuron
- Gradient directions for training

ML Roles

- Feature vector: numeric description of one example
- Weight vector: parameters of linear model/neuron
- Gradient vector: direction to update during training
- Arrow from origin; length = magnitude, direction = pattern

FEATURE AND WEIGHT VECTORS

```
feature_vector = np.array([25, 1.5, 12])   # age, time_on_app, clicks
weight_vector = np.array([0.1, 2.0, -0.5])  # how strongly each feature contributes
print("Feature:", feature_vector)  # One user's behavior in 3D space
```



al

Creating Vectors in NumPy — Basics

np.array, np.zeros, np.ones, np.random

Creation Patterns

- From Python lists for specific values
- All-zeros/ones for initialization
- Random vectors for testing and simulation

ML Best Practices

- np.zeros: initializing gradient accumulators
- np.ones: baseline or mask operations
- Avoid all-zeros weights where symmetry needs breaking

VECTOR CREATION

```
v1 = np.array([1.0, 2.0, 3.0])  # From list: specific values
v_zero = np.zeros(4)            # Zero vector: param initialization
v_ones = np.ones(5)             # Ones vector: baseline/mask
v_rand = np.random.randn(3)     # Random: testing, simulations
```

al

Vector Shapes — (n,) vs (n,1) vs (1,n)

Shape determines broadcasting and matrix multiplication behavior

Shape Basics

- np.array([1,2,3]) → shape (3,) — 1D vector
- np.array([[1],[2],[3]]) → shape (3,1) — column
- np.array([[1,2,3]]) → shape (1,3) — row
- (3,) is most common for vectors in NumPy

Why Shapes Matter

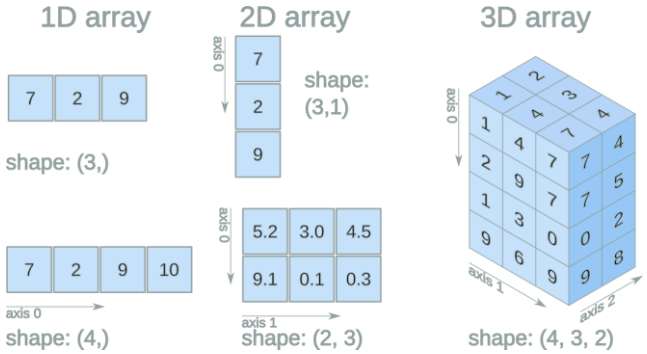
- Determines how operations broadcast
- Affects matrix multiplication behavior
- Many ML bugs from unintended shapes
- Confusing (n,) and (n,1) causes subtle bugs

SHAPE EXAMPLES

```
v_1d = np.array([1, 2, 3]);  print("1D:", v_1d.shape)     # (3,)
v_col = np.array([[1],[2],[3]]); print("Col:", v_col.shape) # (3, 1)
v_row = np.array([[1,2,3]]); print("Row:", v_row.shape)     # (1, 3)
```

`(n,)` = a **1D array** (a vector)

`(n,1)` = a **2D array** with **one column** (a column matrix)



al

Vector Addition — Combining Effects

Component-wise combination of vectors

Intuitive Meaning

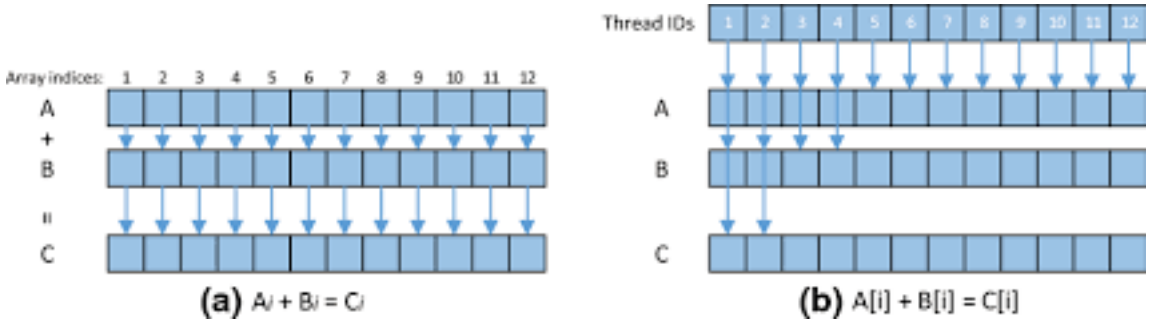
- Adds components feature-wise
- Combines contributions from different sources
- Vectors must be same shape (or broadcastable)

ML Relevance

- Gradient descent update: w_new = w_old - lr * grad
- v_a + v_b: if v_a is weights, v_b is update, result is new weights
- Residual connections add feature maps

VECTOR ADDITION

```
v_a = np.array([1.0, 2.0, 3.0])
v_b = np.array([0.5, -1.0, 0.0])
v_sum = v_a + v_b  # [1.5, 1.0, 7.0] - component-wise
print("v_sum:", v_sum, "shape:", v_sum.shape)
```



al

Scalar Multiplication — Scaling Vectors

Multiply vector by scalar to change length/direction

Intuitive Meaning

- Small positive: shrinks vector (shorter arrow)
- Large positive: stretches vector
- Negative: flips direction and scales

ML Relevance

- Learning rate: scalar controlling step size
- lr * gradient = actual weight update magnitude
- Smaller lr → smaller steps → slower but stable

SCALAR MULTIPLICATION

```
v = np.array([2.0, -1.0, 3.0])
v_small = 0.1 * v   # [0.2, -0.1, 0.3] - same direction, shorter
v_neg = -2.0 * v    # [-4.0, 2.0, -6.0] - opposite direction, longer
# Learning rate scales gradient: w = w - lr * grad
```

al

Vector Norms — Length of a Vector

Measures the 'size' or 'length' in feature space

Norm Definition

- L2 norm: Euclidean length (most common in ML)
- $||v|| = \text{sqrt}(v_1^2 + v_2^2 + ... + v_n^2)$
- Larger norm → vector is 'longer' in feature space

ML Relevance

- L2 regularization: penalty ∝ weight norm
- Gradient clipping limits gradient norms
- Feature scaling controls feature norms

COMPUTING NORMS

```
v = np.array([3.0, 4.0])
norm_v = np.linalg.norm(v)  # L2 norm by default
print("||v||:", norm_v)      # 5.0 (classic 3-4-5 triangle)
# L2 regularization loss += lambda * ||w||²
```

$$u = \begin{bmatrix} u_1 \\ u_2 \\ \cdots \\ u_n \end{bmatrix}$$

$$||u||_2 = \sqrt{u_1^2 + u_2^2 + \cdots + u_n^2}$$

al

Norms of Feature and Weight Vectors

Why controlling norms matters in ML

Feature Vectors

- Norm = overall magnitude of features
- Large norms dominate model outputs if not scaled
- Normalize features to control feature norms

Weight Vectors

- Norm reflects model complexity/sensitivity
- Large weight norms → overfitting, unstable outputs
- Use regularization to control weight norms

FEATURE AND WEIGHT NORMS

```
feature = np.array([10.0, 0.1, 0.5])  # Different scales!
weights = np.array([0.2, 5.0, -3.0])
print("||feature||:", np.linalg.norm(feature))  # Large if unscaled
print("||weights||:", np.linalg.norm(weights))  # Regularization target
```

al

DAY 37

Dot Product

Similarity, Projection, and Model Scoring

OBJECTIVES

- Understand dot product as alignment measure
- Difference from element-wise multiplication
- Compute with np.dot and @ operator
- Connect to linear models and cosine similarity
- Interpret zero dot product (orthogonality)

ACTIVITY

Compute dot products of vector pairs.

Interpret alignment and model scores.

ASSESSMENT

Short answer: What does dot product of zero indicate?

al

Intuitive Meaning

How much two vectors point in the same direction

Vector Intuition

- Large positive: strong alignment (same direction)
- Large negative: opposite directions
- Zero: orthogonal (no alignment)
- Measures 'how much' vectors agree

ML Intuition

- Features aligned with weights → large positive score
- Features opposite to weights → negative score
- Zero: feature pattern is 'neutral' to weights

DOT PRODUCT EXAMPLE

```
v1 = np.array([1.0, 2.0, 3.0])
v2 = np.array([0.5, 1.0, 1.5])  # Scaled version of v1
dot_v1_v2 = np.dot(v1, v2)       # Strong alignment → positive
```

Dot product is:

$$v1 \cdot v2 = \sum_i v1_i \, v2_i$$

So here:

- 1.0 * 0.5 = 0.5
- 2.0 * 1.0 = 2.0
- 3.0 * 1.5 = 4.5

Total:

$$0.5 + 2.0 + 4.5 = 7.0$$

al

Dot Product vs Element-wise Multiplication

Vector result vs scalar result

Element-wise

- v1 * v2: multiply each component independently
- Result is another vector (same shape)
- [1,2,3] * [4,5,6] = [4, 10, 18]

Dot Product

- Multiplies and sums into a scalar
- dot(v1, v2) = 4 + 10 + 18 = 32
- Linear model output is weighted sum (dot product)

COMPARISON

```
v1 = np.array([1.0, 2.0, 3.0]); v2 = np.array([4.0, 5.0, 6.0])
elementwise = v1 * v2  # [4, 10, 18] - keeps components
dot_result = np.dot(v1, v2)  # 32 - single score
# Linear model: score = dot(weights, features), not element-wise
```

al

Computing with np.dot and @ Operator

Two equivalent syntaxes for dot product

NumPy APIs

- np.dot(a, b): vector dot product
- a @ b: equivalent for 1D vectors
- @ also works for matrix multiplication (2D)

ML Connection

- Model output: w @ x + b
- This is exactly linear/logistic regression before activation
- Understanding clarifies how features contribute

NP.DOT VS @

```
w = np.array([0.2, -0.5, 1.0])  # weights
x = np.array([10.0, 2.0, 0.5])   # features
score_dot = np.dot(w, x)  # 1.5
score_at = w @ x          # 1.5 - same result
```

al

Cosine Similarity

Normalized dot product measuring angle

Definition

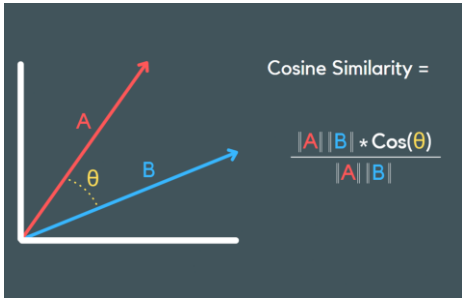
- cos(a, b) = dot(a, b) / (||a|| × ||b||)
- Value ~1: nearly identical direction
- Value ~0: orthogonal (perpendicular)
- Value ~−1: opposite directions

ML Relevance

- Information retrieval: document similarity
- Embeddings: word/sentence similarity
- Recommendation systems: user/item matching

COSINE SIMILARITY

```
def cosine_similarity(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
u = np.array([1.0, 0.0]); v = np.array([0.0, 1.0])
print("cos(u, v):", cosine_similarity(u, v))  # 0.0 - orthogonal
```



al

Dot Product in Linear Models

Core computation: weighted sum of features

Linear Scoring

- score = w @ x + b (bias term)
- Dot product computes weighted feature sum
- Each weight determines feature contribution

Interpretation

- Positive weight + positive feature → increases score
- Negative weight + positive feature → decreases score
- Understanding dot product = understanding models

LINEAR MODEL SCORING

```
w = np.array([0.3, 1.5, -2.0]); b = 0.5
x1 = np.array([1.0, 0.0, 0.0])  # Only first feature active
x2 = np.array([0.0, 1.0, 0.0])  # Only second feature active
print("score(x1):", w @ x1 + b, "score(x2):", w @ x2 + b)
```

al

Orthogonality — Zero Dot Product

Vectors with no directional overlap

Meaning

- dot(a, b) = 0 → perpendicular vectors
- No directional overlap in feature space
- One lies completely outside the other's direction

ML Relevance

- Orthogonal features are 'independent' in linear combinations
- Decorrelated features simplify optimization
- Used to enforce diversity in learned representations

ORTHOGONALITY EXAMPLE

```
a = np.array([1.0, 0.0]); b = np.array([0.0, 2.0])
print("dot(a, b):", np.dot(a, b))  # 0 - orthogonal!
c = np.array([1.0, 1.0])
print("dot(a, c):", np.dot(a, c))  # 1 - some alignment
```

al

Common Shape Errors

Mismatched lengths and confusing shapes

Typical Issues

- Mismatched lengths: np.dot([1,2,3], [1,2]) → error
- Confusing (n,) vs (n,1) in matrix-vector products
- Dot product requires same length vectors

Best Practices

- Always inspect .shape before dot products
- Reshape to clarify: .reshape(-1, 1) or .reshape(1, -1)
- Especially important in batch operations

SHAPE ERROR EXAMPLE

```
a = np.array([1.0, 2.0, 3.0]); b = np.array([1.0, 2.0])
try: np.dot(a, b)  # Error: shapes (3,) and (2,) not aligned
except ValueError as e: print("Error:", e)
col_vec = a.reshape(-1, 1)  # (3, 1) for matrix operations
```

al

DAY 38

Matrices

Collections of Vectors for ML

OBJECTIVES

- Understand matrices as 2D arrays (rows × columns)
- Interpret rows as samples, columns as features
- Practice creation, shape inspection, transpose
- Learn matrix multiplication rules and shapes
- Distinguish matrix vs element-wise multiplication

ACTIVITY

Create matrices and perform basic operations.

Compute X @ W and interpret shapes.

ASSESSMENT

Mini-quiz on matrix shapes, transposes, multiplication.

al

What Matrices Represent in ML

2D arrays for data and parameters

Matrix Intuition

- 2D array of numbers (rows × columns)
- Rows often = samples; columns = features
- Row 0 = feature vector for sample 1
- Column 1 = values of feature 2 across samples

ML Roles

- Data matrix X: shape (n_samples, n_features)
- Weight matrix W: (n_features, n_outputs)
- Mini-batches: subset of rows from full data

DATA MATRIX EXAMPLE

```
X = np.array([[1.0, 2.0, 3.0],   # sample 1
              [4.0, 5.0, 6.0],   # sample 2
              [7.0, 8.0, 9.0]])  # sample 3
print("X shape:", X.shape)  # (3, 3) - 3 samples, 3 features
```

al

Creating Matrices and Inspecting Shapes

From nested lists and random generators

Creation Patterns

- From nested Python lists
- Using random generators for testing
- np.zeros((rows, cols)), np.ones((rows, cols))

Shape Interpretation

- W shape (3, 2): 3 input features, 2 outputs
- X @ W transforms feature dim=3 into output dim=2
- Always annotate shapes in comments!

WEIGHT MATRIX CREATION

```
W = np.array([[0.1, -0.2],    # weights: feature1 → outputs
              [0.4, 0.0],     # feature2 → outputs
              [-0.3, 0.5]])   # feature3 → outputs
print("W shape:", W.shape)    # (3, 2) - 3 inputs, 2 outputs
```

al

Matrix Addition — Same Shape Required

Element-wise combination of matrices

Concept

- Element-wise addition: each entry added
- Requires same shape (or broadcastable)
- Common uses: adding biases or residuals

ML Relevance

- Residual connections: output + input
- Adding bias matrices to layer outputs
- Must have compatible shapes!

MATRIX ADDITION

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[10, 20], [30, 40]])
C = A + B  # [[11, 22], [33, 44]] - element-wise
print("C shape:", C.shape)  # Same as inputs: (2, 2)
```

al

Matrix Transpose — Swapping Rows and Columns

Flip matrix over its diagonal

Definition

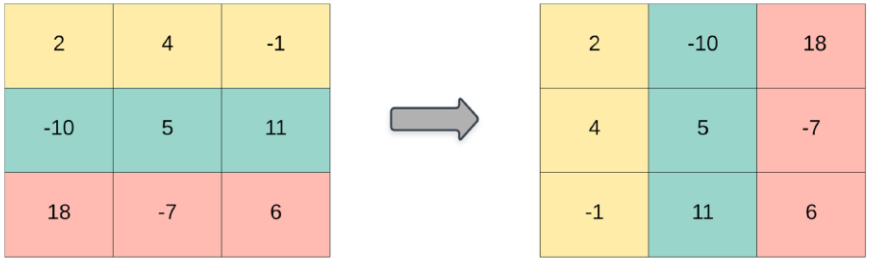
- Rows become columns, columns become rows
- X.T: X transposed
- (2, 3) matrix → (3, 2) after transpose

Why It Matters

- Align shapes for multiplication
- Some algorithms expect features as rows
- Notation conventions vary: row vs column vectors

TRANSPOSE EXAMPLE

```
X = np.array([[1, 2, 3], [4, 5, 6]])  # (2, 3)
X_T = X.T                              # (3, 2)
print("X shape:", X.shape, "X_T shape:", X_T.shape)
# Transpose switches roles of features and samples
```



al

Matrix Multiplication Rules and Shapes

Inner dimensions must match

Shape Rule

- A (m×n) @ B (n×p) → result (m×p)
- Inner dimensions must match (n)
- Each output entry = dot product of A row × B column

ML Mapping

- X (samples×features) @ W (features×outputs)
- → predictions (samples×outputs)
- Core of linear layers and linear regression

MATRIX MULTIPLICATION

```
X = np.array([[1.0, 2.0, 3.0], [4.0, 5.0, 6.0]])  # (2, 3)
W = np.array([[0.1, -0.2], [0.4, 0.0], [-0.3, 0.5]]) # (3, 2)
Y = X @ W  # (2, 2) - 2 samples, 2 outputs
```

al

Matrix vs Element-wise Multiplication

@ operator vs * operator

Element-wise (*)

- A * B: multiply corresponding entries
- Requires same shape or broadcastable
- Result has same shape as inputs

Matrix Product (@)

- A @ B: dot products of rows with columns
- Different shape rule (inner dims match)
- Mixes information across dimensions

COMPARISON

```
A = np.array([[1, 2], [3, 4]]); B = np.array([[10, 20], [30, 40]])
elementwise = A * B  # [[10, 40], [90, 160]] - entry-wise
matmul = A @ B       # [[70, 100], [150, 220]] - row × column dots
# Weight application uses @, not element-wise *
```

al

Common Matrix Shape Errors

Incompatible inner dimensions

Error Types

- Incompatible inner dimensions for multiplication
- Confusing which dim is samples vs features
- (2, 3) @ (2, 2) fails: 3 ≠ 2

Best Practices

- Always check shapes before @
- Annotate code: # (samples × features × outputs)
- Use .T if dimensions need alignment

SHAPE ERROR EXAMPLE

```
X_bad = np.array([[1, 2, 3], [4, 5, 6]])  # (2, 3)
W_bad = np.array([[0.1, 0.2], [0.3, 0.4]]) # (2, 2)
try: X_bad @ W_bad  # Error: 3 vs 2 mismatch
except ValueError as e: print("Error:", e)
```

al

DAY 39

Broadcasting

Vectorized Computation without Loops

OBJECTIVES

- Understand broadcasting as implicit array expansion
- Learn broadcasting rules (align from right, size 1 expands)
- Apply for row-wise and column-wise operations
- Recognize when broadcasting fails or goes wrong
- Use broadcasting for feature standardization

ACTIVITY

Normalize a matrix by row using broadcasting.

No explicit Python loops.

ASSESSMENT

Code challenge: correct broadcasting in ML operation.

al

What Broadcasting Is and Why It Exists

Implicit expansion to match shapes

Intuitive Idea

- NumPy treats smaller arrays as if repeated
- Avoids manual loops or explicit replication
- Enables concise, fast vectorized operations

ML Relevance

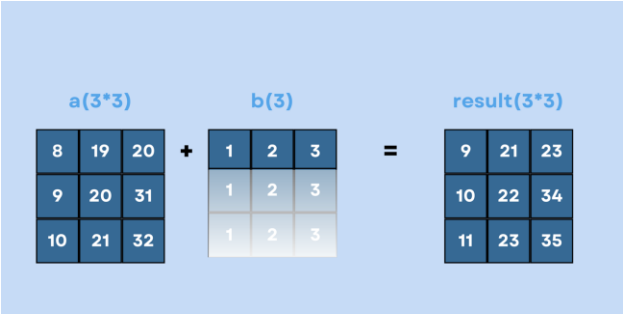
- Apply operations across batches with one line
- Critical for performance in ML pipelines
- Adding bias to every sample uses broadcasting

BASIC BROADCASTING

```
v = np.array([1.0, 2.0, 3.0])        # (3,)
M = np.array([[10., 20., 30.], [40., 50., 60.]])  # (2, 3)
result = M + v  # v stretched to (2, 3), added to each row
# Adds bias to every sample: X + bias
```

```
[[1, 2, 3],
 [1, 2, 3]]
```

```
result =
[[11., 22., 33.],
 [41., 52., 63.]]
```



al

Broadcasting Rules (Simplified)

Align from right, expand size-1 dimensions

Key Rules

- Align shapes from the right (trailing dims)
- Dimensions must be equal OR one must be 1
- Size-1 dimension expands to match other

Valid Examples

- (2, 3) + (3,) → (2, 3)
- (4, 1) + (4, 5) → (4, 5)
- (3, 1) + (1, 4) → (3, 4)

BROADCASTING RULES

```
A = np.ones((4, 1)); B = np.arange(20).reshape(4, 5)
C = A + B  # (4, 1) broadcasts to (4, 5)
print("A shape:", A.shape, "B shape:", B.shape, "C shape:", C.shape)
```

al

Row-wise vs Column-wise Operations

axis=0 vs axis=1 with keepdims=True

Row-wise (axis=1)

- Operate across columns for each row
- Mean per row: shape (n, 1)
- Broadcasts across columns

Column-wise (axis=0)

- Operate across rows for each column
- Mean per column: shape (1, m)
- Broadcasts down rows - feature standardization

ROW VS COLUMN

```
X = np.array([[1., 2., 3.], [4., 5., 6.], [7., 8., 9.]])
row_means = X.mean(axis=1, keepdims=True)  # (3, 1)
col_means = X.mean(axis=0, keepdims=True)  # (1, 3)
X_centered_cols = X - col_means  # Feature standardization
```

```
col_means =
[[4. 5. 6.]]
```

```
row_means =
[[2.]
 [5.]
 [8.]]
```

al

When Broadcasting Fails

Shape incompatibility errors

Failure Conditions

- After alignment, neither dim is 1 and they differ
- (2, 3) + (4, 1): 2 vs 4 and 3 vs 1 incompatible
- Raises ValueError with shape mismatch

Best Practices

- Inspect shapes before broadcasting
- Don't assume it 'just works'
- Print shapes during debugging

BROADCASTING FAILURE

```
A = np.ones((2, 3)); B = np.ones((4, 1))
try: C = A + B  # (2, 3) vs (4, 1) - incompatible
except ValueError as e: print("Broadcasting error:", e)
# 3 vs 1 OK, but 2 vs 4 fails
```

al

Feature Standardization with Broadcasting

Column-wise mean=0, std=1

Steps

- Compute mean and std per column (axis=0)
- Subtract mean, divide by std using broadcasting
- Result: each column mean~0, std~1

ML Relevance

- Improves numerical stability
- Required for many models (gradient descent)
- Same transformation applied at inference time

STANDARDIZATION

```
X = np.array([[1., 10.], [2., 20.], [3., 30.]])
means = X.mean(axis=0, keepdims=True)  # (1, 2)
stds = X.std(axis=0, keepdims=True)    # (1, 2)
X_std = (X - means) / stds  # Each column: mean=0, std=1
```

al

DAY 40

Matrix Operations Exercise

Putting It All Together

OBJECTIVES

- Implement linear transformations with matrix multiplication
- Use broadcasting to add biases and normalize
- Avoid explicit Python loops over samples
- Diagnose and fix shape errors in ML code
- Build a complete transformation pipeline

ACTIVITY

Build a linear transformation pipeline in NumPy.

Standardize → Linear Layer → Normalize.

ASSESSMENT

Graded notebook: correct operations, shapes, explanations.

al

DAY 40 · EXERCISE

Linear Transformation as Matrix Multiplication

Y = X @ W + b — the core ML computation

Concept

- Maps input vectors to output vectors
- Single sample: y = W @ x + b
- Batch: Y = X @ W + b (bias broadcasts)

ML Relevance

- Core of dense layers in neural networks
- Linear regression prediction
- Each row of Y = transformed sample

BATCH LINEAR TRANSFORMATION

```
X = np.array([[1., 0.5], [2., -1.], [0., 3.]])  # (3, 2)
W = np.array([[0.2, -0.1, 0.5], [0.7, 0.3, -0.2]]) # (2, 3)
b = np.array([0.1, 0.0, -0.3])  # (3,)
Y = X @ W + b  # (3, 3) - 3 samples, 3 outputs
```

al

DAY 40 · EXERCISE

Vectorized vs Loop Implementation

Why vectorization matters

Naive Loop

- For each sample: compute W @ x_i + b
- Slow in Python (interpreter overhead)
- Verbose and error-prone code

Vectorized

- Compute X @ W + b once using NumPy
- Fast (optimized C under the hood)
- Clean, readable code

LOOP VS VECTORIZED

```
def linear_loop(X, W, b):
    return np.vstack([x @ W + b for x in X])  # Slow!

Y_vec = X @ W + b  # Fast! Same result
print("Diff:", np.linalg.norm(linear_loop(X, W, b) - Y_vec))
```

al

DAY 40 · EXERCISE

Debugging Shape Errors in Linear Layers

Common mistakes and fixes

Common Errors

- Switching X and W: W @ X instead of X @ W
- Wrong bias shape: (batch,) instead of (outputs,)
- Inner dimensions don't match

Debugging Strategy

- Print shapes at each step
- Annotate expected shapes in comments
- Use .shape before every @ operation

DEBUGGING SHAPES

```
try: wrong_Y = W @ X  # Order matters! W(2,3) @ X(3,2) fails
except ValueError as e: print("Error:", e)
wrong_b = np.array([[0.1], [0.0], [-0.3]])  # (3, 1) not (3,)
# Always check: X(batch, features) @ W(features, outputs) + b(outputs)
```

al

DAY 40 · EXERCISE

Broadcasting with Norm-Based Scaling

Combining operations in a pipeline

Example

- Normalize outputs per feature using broadcasting
- Each output dimension: mean~0, std~1
- Can feed into another layer or visualize

Implementation

- Y_mean = Y.mean(axis=0, keepdims=True)
- Y_std = Y.std(axis=0, keepdims=True)
- Y_norm = (Y - Y_mean) / (Y_std + 1e-8)

NORM-BASED SCALING

```
Y_mean = Y.mean(axis=0, keepdims=True)  # (1, 3)
Y_std = Y.std(axis=0, keepdims=True)    # (1, 3)
Y_norm = (Y - Y_mean) / (Y_std + 1e-8)  # Stabilize with epsilon
# Each output column now has mean~0, std~1
```

al

DAY 40 · EXERCISE

Matrix Operations for Feature Engineering

Linear combinations as feature transforms

Concept

- X_new = X @ A: linear combination of features
- Project high-dimensional to lower-dimensional
- Many feature transforms are matrix operations

Example

- 3 features → 2 derived features
- A encodes how to combine originals
- PCA, projections use this pattern

FEATURE TRANSFORMATION

```
X_raw = np.array([[1., 2., 3.], [0.5, 1.5, 2.], [2., 0.5, 1.]])
A = np.array([[1., 0.], [0.5, 0.5], [0., 1.]])  # (3, 2)
X_new = X_raw @ A  # (3, 2) - 2 derived features
print("X_raw:", X_raw.shape, "→ X_new:", X_new.shape)
```

al

DAY 40 · ACTIVITY

Linear Transformation Pipeline

End-to-end pipeline without loops

Pipeline Steps

1. Create X (batch_size, n_features)
2. Create W and b with compatible shapes
3. Standardize X per feature (column-wise)
4. Compute Y = X_standardized @ W + b
5. Optionally normalize rows of Y to unit norm
6. Print shapes at each step
7. No Python loops!

Assessment Questions

Q1: Explain X @ W + b in words?

Q2: How to normalize rows to unit length?

Q3: Advantage of vectorized over loops?

Q4: Example shape error and how to fix?

Q5: How to debug wrong prediction shapes?

al

Week 8 Complete!

Math for ML I — Linear Algebra Foundations

Vectors	Ordered lists; shapes (n,) vs (n,1); addition, scalar mult, norms; feature/weight vectors
Dot Product	Alignment measure; cosine similarity; linear model scoring; orthogonality = zero
Matrices	2D arrays (samples×features); transpose; X @ W for linear transformation
Broadcasting	Implicit expansion; axis=0 vs axis=1; feature standardization; row normalization
Exercise	Y = X @ W + b pipeline; vectorized over loops; debugging shapes

Key Takeaway
Linear algebra is the language of ML. Understanding vectors, dot products, matrices, and broadcasting enables you to read, write, and debug ML code effectively.

THE ML LINEAR ALGEBRA CHAIN

```
X_std = (X - X.mean(0, keepdims=1)) / X.std(0, keepdims=1)  # Standardize
Y = X_std @ W + b  # Linear transform  # → predictions in one line, no loops
```

al